

ESTUDO DIRIGIDO — UNIDADE IV

Aplicações em Sistemas Web com Java



Instruções Gerais

Este estudo dirigido é uma trilha de aprendizagem autoguiada dividida em 5 trilhas temáticas.

Cada trilha apresenta conceitos, atividades práticas cronometradas (🕒) e checkpoints (🏁).

Execute as atividades na ordem proposta — cada uma prepara para a seguinte.

Tempo total estimado: 300 minutos (5 horas de estudo dirigido).

Material de apoio: Apostila Unidade IV, documentação Spring Boot, JavaDocs do JCF.

TRILHA 1 — Benchmarking com JMH (60 min)

Objetivo: Configurar e executar benchmarks JMH; interpretar resultados e identificar diferenças reais de desempenho entre estruturas do JCF.

1.1 Conceitos Essenciais

Antes de executar qualquer benchmark, é fundamental entender o que torna uma medição confiável. Responda mentalmente:

- Por que o compilador JIT invalida medições ingênuas?
- O que é "dead code elimination" e como o Blackhole do JMH previne?
- Por que usar `@Fork(2)` em vez de executar na mesma JVM?

Atividade 1.1 — Benchmark Básico — HashMap vs TreeMap ⌚ 15 min

Crie um projeto Maven com as dependências do JMH (seção 4.1 da Apostila).

Copie a classe `MapBenchmark` da seção 4.2 e execute com: `mvn clean package && java -jar target/benchmarks.jar`

Registre os resultados no quadro abaixo:

hashMapGet: _____ ns/op (±_____ erro)

treeMapGet: _____ ns/op (±_____ erro)

treeMapRange: _____ ns/op (±_____ erro)

Orientação:

Esperado: hashMapGet < treeMapGet por fator de 3–10x.

A variância (erro) deve ser < 10% para uma medição confiável.

O range scan do TreeMap é seu diferencial real — HashMap não oferece essa operação.

Atividade 1.2 — Adicionando ArrayList vs LinkedList ⌚ 20 min

Adicione ao `MapBenchmark` dois novos métodos:

- `arrayListGet`: `ArrayList` com 10.000 inteiros, mede `get(índice_do_meio)`

- `linkedListGet`: `LinkedList` com 10.000 inteiros, mede `get(índice_do_meio)`

Qual diferença de desempenho você espera? Anote antes de executar:

Previsão: _____

Execute e registre o resultado:

arrayListGet: _____ ns/op

LinkedListGet: _____ ns/op

Razão (linked/array): _____x mais lento

 Orientação:

O `LinkedList.get(i)` é $O(n)$ — percorre a lista até o índice.

O `ArrayList.get(i)` é $O(1)$ — acesso direto ao array interno.

Para lista de 10.000, espera-se razão de ~5.000x (acesso ao meio da lista).

 Atividade 1.3 — Interpretando GC Logs ⌚ 25 min

Execute sua aplicação Spring Boot (ou um loop simples) com os flags:

```
java -Xlog:gc*:file=gc.log:time,uptime -jar app.jar
```

Analise o gc.log e responda:

- a) Qual o tempo médio entre coletas? _____
- b) Qual a duração máxima de pausa? _____
- c) As pausas afetariam uma API com SLA de 200ms? _____
- d) Que ajuste de heap reduziria a frequência de GC? _____

 Orientação:

Frequência elevada de GC (< 1s entre coletas) indica heap pequeno ou objetos de vida curta excessivos.

Pausas > 200ms violam SLAs de APIs REST. G1GC com `-XX:MaxGCPauseMillis=200` melhora a situação.

Aumentar `-Xmx` reduz frequência; usar objetos de vida longa (cache) reduz pressão no Young Gen.

 Checkpoint 1

1. Qual estrutura foi mais rápida no benchmark: HashMap ou TreeMap? Por quê?
2. Em qual cenário o TreeMap superaria o HashMap na prática?
3. O que aconteceria ao benchmark se você remover `@Fork`? Por quê isso é problemático?
4. Quais flags JVM você usaria para monitorar GC em produção?

TRILHA 2 — Estruturas em Bancos de Dados (45 min)

Objetivo: Compreender como árvores B+ implementam índices de banco de dados e aplicar esse conhecimento com JPA/Hibernate.

2.1 Árvores B+ na Prática

Atividade 2.1 — Analisando o Plano de Execução SQL ⌚ 15 min

Crie a entidade `transacoes` com `@Index` conforme a Seção 3.2.3 da Apostila.

Configure o H2 Console (`spring.h2.console.enabled=true`).

Execute a query: `EXPLAIN SELECT * FROM transacoes WHERE conta_id = 42`

Compare com: `EXPLAIN SELECT * FROM transacoes WHERE valor > 100`

Query com índice (`conta_id`): _____

Query sem índice (`valor`): _____

Diferença observada: _____

Orientação:

Com índice: *"INDEX SCAN on idx_conta_id" — percorre a árvore B+.*

Sem índice: *"TABLE SCAN" ou "FULL SCAN" — percorre todas as linhas.*

Para tabelas grandes, a diferença é de milissegundos vs segundos.

Atividade 2.2 — Detectando o Problema N+1 ⌚ 15 min

Crie um endpoint que retorna 20 transações com seus usuários associados (relacionamento `@ManyToOne`).

Habilite: `spring.jpa.show-sql=true` no `application.properties`.

Acesse o endpoint e conte quantas queries SQL foram executadas:

Número de queries: _____

Esperado sem N+1: _____

Corrija o problema usando `@EntityGraph` ou `JOIN FETCH` e conte novamente:

Número de queries após correção: _____

Orientação:

Sem correção: *1 query para listar transações + N queries para buscar cada usuário = N+1.*

Com `@EntityGraph` ou `JOIN FETCH`: *1 única query com JOIN, sem queries adicionais.*

Este é o bug de performance mais comum em projetos Hibernate.

Atividade 2.3 — Análise de Altura da Árvore B+ ⌚ 15 min

Use a fórmula: $\text{altura} = \text{ceil}(\log_m(n))$ para calcular:

m = 100 filhos por nó, n = 1.000.000 registros: h = _____

m = 500 filhos por nó, n = 1.000.000 registros: h = _____

m = 500 filhos por nó, n = 1.000.000.000 registros: h = _____

O que significa altura h em termos de operações de disco?

Resposta: _____

Por que SGBDs escolhem m alto (centenas)?

Resposta: _____

 Orientação:

$h = \text{ceil}(\log_{100}(10^6)) = 3$ leituras de disco no pior caso.

$h = \text{ceil}(\log_{500}(10^9)) \approx 3$ leituras para um bilhão de registros.

m alto = cada nó cabe em uma página de disco (4KB–16KB), minimizando I/O.

Checkpoint 2

1. Qual a diferença entre árvore B e árvore B+?
2. Por que as folhas da árvore B+ são ligadas em lista encadeada?
3. O que é o problema N+1 no Hibernate e como corrigi-lo?
4. Que tipo de query se beneficia mais de um índice B+: busca pontual ou range scan?

TRILHA 3 — Caching com Spring Cache e Redis (60 min)

Objetivo: Implementar cache eficiente com Spring Cache, LRU Cache e Redis; medir o impacto no desempenho.

3.1 Spring Cache na Prática

Atividade 3.1 — Configurando Spring Cache com Ehcache (local) ⌚ 20 min

Adicione spring-boot-starter-cache ao seu pom.xml.

Anote a Application com @EnableCaching.

Implemente um SaldoService com:

- buscarSaldo(Long id): @Cacheable(value="saldos", key="#id")
- atualizarSaldo(Conta c): @CachePut(value="saldos", key="#c.id")
- invalidarSaldo(Long id): @CacheEvict(value="saldos", key="#id")

Adicione um contador de chamadas ao banco (AtomicLong) para medir cache hits/misses.

Faça 5 chamadas buscarSaldo(1L) e verifique quantas chegaram ao banco:

Chamadas ao banco: _____ (esperado: 1 — a primeira)

Orientação:

Apenas a primeira chamada deve chegar ao banco; as 4 seguintes retornam do cache.

O contador AtomicLong deve marcar 1 chamada ao banco em 5 invocações.

@CachePut garante atualização simultânea do cache e do banco na escrita.

Atividade 3.2 — Implementando LRU Cache com LinkedHashMap ⌚ 20 min

Implemente a classe LRUCache<K,V> conforme a Seção 3.3.3 da Apostila.

Crie um cache com capacidade = 3 e execute a seguinte sequência:

put(A, 1) → put(B, 2) → put(C, 3) → get(A) → put(D, 4)

Qual elemento foi eviccionado? _____ (esperado: B)

Por que A não foi eviccionado, apesar de ter sido inserido antes de B?

Resposta: _____

Adicione logging ao removeEldestEntry para confirmar qual elemento é removido.

Orientação:

B é eviccionado porque, após get(A), A torna-se o mais recentemente acessado.

A ordem de evicção após `get(A)`: *B (mais antigo) → C → A (mais recente)*.

O parâmetro `accessOrder=true` no construtor do `LinkedHashMap` habilita esse comportamento.

Atividade 3.3 — Benchmark: Cache Local vs Banco de Dados ⌚ 20 min

Configure um JMH benchmark com dois métodos:

- `buscarSemCache`: sempre consulta o banco (ou simula com `Thread.sleep(5ms)`)
- `buscarComCache`: usa o `LRUCache` (retorno imediato se em cache)

Execute com 100% de cache hits (todos os IDs já estão no cache).

Registre:

sem cache: _____ ns/op

com cache: _____ ns/op

speedup: _____ x

O que acontece ao speedup se a taxa de cache hit cair para 50%?

Resposta: _____

Orientação:

Com 100% de hits, o speedup deve ser de 100–10.000x (memória vs disco).

Com 50% de hits, o speedup é reduzido pela metade — o custo de cada miss é uma query lenta.

Isso motiva o dimensionamento correto do cache: maior capacidade = maior hit rate.

Checkpoint 3

1. Qual a diferença entre `@Cacheable` e `@CachePut`?
2. O que acontece se você esquecer o `@CacheEvict` ao atualizar um registro no banco?
3. Por que o LRU Cache usa `accessOrder=true` no `LinkedHashMap`?
4. Em que situação você escolheria Redis em vez de `LRUCache` com `LinkedHashMap`?

TRILHA 4 — Mensageria e Padrões de Uso (60 min)

Objetivo: Implementar processamento assíncrono com RabbitMQ; aplicar Rate Limiting e Trie em cenários reais.

4.1 RabbitMQ — Processamento Assíncrono

Atividade 4.1 — Configurando RabbitMQ com Docker ⌚ 20 min

Suba o RabbitMQ com Docker: `docker run -d -p 5672:5672 -p 15672:15672 rabbitmq:3-management`

Acesse o painel de administração: `http://localhost:15672 (guest/guest)`

Configure as dependências no `pom.xml`: `spring-boot-starter-amqp`

Implemente a configuração `RabbitMQConfig` com `Exchange`, `Queue` e `Binding` (Seção 3.4.2).

Implemente um `Producer` que envia uma mensagem a cada `POST /test/mensagem`

Implemente um `Consumer` com `@RabbitListener` e adicione log da mensagem recebida.

Observe no painel do RabbitMQ o fluxo das mensagens.

Número de mensagens processadas após 5 POSTs: _____

Orientação:

O painel deve mostrar 5 mensagens publicadas e 5 entregues.

O log do Consumer deve exibir 5 mensagens recebidas em thread separada.

Observe que o endpoint `POST` retorna 200 antes do Consumer processar — isso é o desacoplamento.

Atividade 4.2 — Rate Limiter — Sliding Window ⌚ 20 min

Implemente o `RateLimiter` da Seção 3.5.1 (Sliding Window com Deque).

Integre ao `RateLimitFilter` (`OncePerRequestFilter`).

Configure o limite para 10 requisições por minuto (para facilitar os testes).

Faça 10 requisições consecutivas ao endpoint `/api/produtos/1`:

Todas retornaram 200? _____

Faça a 11ª requisição:

Código HTTP retornado: _____ (esperado: 429 Too Many Requests)

Aguarde 1 minuto e faça a 12ª requisição:

Código HTTP: _____ (esperado: 200 — janela resetou)

Orientação:

As 10 primeiras devem retornar 200; a 11ª deve retornar 429.

Após 1 minuto, os timestamps expiram da janela deslizante e o limite é resetado.

Isso demonstra o funcionamento do Sliding Window: a janela "desliza" no tempo.

Atividade 4.3 — Trie — Autocompletar por Prefixo ⌚ 20 min

Implemente a classe Trie conforme a Seção 3.5.2 da Apostila.

Insira as palavras: java, javascript, jpa, json, spring, spring-boot, redis, rabbitmq

Teste os seguintes prefixos e registre os resultados:

autocompletar("ja", 5) → _____

autocompletar("sp", 5) → _____

autocompletar("re", 5) → _____

autocompletar("xyz", 5) → _____

Meça o tempo de busca com System.nanoTime() para um prefixo de 2 caracteres

com 10.000 palavras inseridas: _____ ns

Com 100.000 palavras inseridas: _____ ns

A complexidade é realmente independente do número de palavras?

Orientação:

"ja": [java, javascript, jpa]; "sp": [spring, spring-boot]; "re": [redis, rabbitmq].

"xyz": [] — lista vazia, retorno imediato.

O tempo de busca deve ser aproximadamente igual para 10K e 100K palavras — $\Theta(|prefixo|)$.

Checkpoint 4

1. Qual a diferença entre Exchange do tipo Direct e Topic no RabbitMQ?
2. O que é uma Dead Letter Queue e quando ela é usada?
3. Por que o Sliding Window Rate Limiter é mais preciso que o Fixed Window?
4. Qual a vantagem da Trie sobre o HashMap<String, List<String>> para autocompletar?

TRILHA 5 — Integração e Preparação para o Projeto (75 min)

Objetivo: Integrar todos os componentes estudados e planejar a arquitetura do Projeto Integrador.

5.1 Análise de Cenários — Escolha de Estruturas

Atividade 5.1 — Tomada de Decisão — Qual estrutura usar? ⌚ 20 min

Para cada cenário, escolha a estrutura mais adequada e justifique:

1. Endpoint que retorna o perfil de um usuário por ID (100K usuários):

Estrutura: _____ Complexidade: _____

Justificativa: _____

2. Endpoint que lista os 10 produtos mais vendidos (atualizado a cada pedido):

Estrutura: _____ Complexidade: _____

Justificativa: _____

3. Funcionalidade de busca de CEP por prefixo (autocompletar):

Estrutura: _____ Complexidade: _____

Justificativa: _____

4. Verificação se um e-mail já está cadastrado (conjunto de e-mails):

Estrutura: _____ Complexidade: _____

5. Cache de sessões com expiração automática (TTL de 30 min):

Estrutura: _____ Justificativa: _____

Orientação:

1. *HashMap<Long, Usuario> ou Redis — busca $\Theta(1)$ por ID.*
2. *PriorityQueue<Produto> (min-heap, tamanho 10) — manter top-10 em $\Theta(\log 10)$.*
3. *Trie — autocompletar por prefixo em $\Theta(|\text{prefixo}|)$.*
4. *HashSet<String> — verificação de pertencimento em $\Theta(1)$.*
5. *Redis com TTL — expira automaticamente; funciona com múltiplas instâncias.*



Atividade 5.2 — Planejamento da Arquitetura — EduMetrics API

🕒 25 min

Desenhe (no papel ou em um editor) a arquitetura do Projeto Integrador EduMetrics API.

O sistema deve incluir:

- API REST Spring Boot com os seguintes endpoints:

GET /alunos/{id}/desempenho — retorna desempenho do aluno

GET /disciplinas/autocompletar?q=... — sugestões de disciplinas

POST /relatorios — solicita geração de relatório PDF (assíncrono)

GET /admin/stats — estatísticas de cache e rate limiting

Para cada componente, responda:

Cache de desempenho: estrutura=_____ TTL=_____ estratégia=_____

Rate Limiting: algoritmo=_____ limite=_____ janela=_____

Autocompletar: estrutura=_____ como carregar os dados=_____

Relatórios: fila=_____ exchange=_____ routing key=_____



Orientação:

Cache: Redis + @Cacheable; TTL sugerido: 5 min (dados mudam raramente).

Rate Limiting: Sliding Window por IP; 100 req/min é um ponto de partida razoável.

Autocompletar: Trie em memória carregada no @PostConstruct via disciplinaRepository.findAll().

Relatórios: Exchange=relatorios.exchange, Fila=relatorios.pdf, RoutingKey=relatorio.pdf



Atividade 5.3 — Implementando o Esqueleto do Projeto Integrador

🕒 30 min

Crie um projeto Spring Boot com a seguinte estrutura de pacotes:

br.edu.iftm.edumetrics

├── controller (REST endpoints)

├── service (lógica de negócio + cache)

├── repository (JPA repositories)

├── messaging (RabbitMQ producer/consumer)

└── cache (LRUCache, Trie)

└─ security (RateLimiter, RateLimitFilter)
└─ config (CacheConfig, RabbitMQConfig)

Implemente o esqueleto (classes com métodos stubados) de todos os componentes.

Crie os testes JUnit 5 para:

- ☐ LRUCacheTest — capacidade máxima e evicção LRU
- ☐ TrieTest — inserção e autocompletar
- ☐ RateLimiterTest — limites e independência entre clientes

Verifique se todos os testes passam antes de implementar a lógica real.

Orientação:

TDD (Test-Driven Development): escrever testes antes da implementação garante especificação clara.

Os testes já implementados na Seção 7 da Apostila são um ponto de partida.

Com mvn test, todos os testes devem passar (pelo menos os stubs que retornam valores padrão).

Checkpoint 5

1. Para uma API com múltiplas instâncias no Kubernetes, por que você usaria Redis em vez de LRUCache local?
2. Qual o impacto de um cache sem TTL em produção após 30 dias de operação?
3. Como você testaria se o Rate Limiter realmente respeita os limites configurados?
4. Qual o benefício do processamento assíncrono com RabbitMQ para a latência percebida pelo usuário?

Síntese da Unidade IV — Mapa Conceitual

Complete o mapa conceitual conectando cada estrutura de dados ao seu caso de uso profissional:

Estrutura de Dados	Caso de Uso	Vantagem Principal
HashMap / Redis	_____	$\Theta(1)$ — acesso direto por chave
LinkedHashMap (LRU)	_____	Evicção automática do item mais antigo
TreeMap	_____	$\Theta(\log n)$ — mantém ordenação
Trie	_____	$\Theta(\text{prefixo})$ — busca por prefixo
Deque (Sliding Window)	_____	Janela deslizando para rate limiting
PriorityQueue (Heap)	_____	$\Theta(\log k)$ — manter top-K elementos

Preparação para o Projeto Integrador — Checklist Final

- ☐ Sei configurar JMH e interpretar resultados de benchmarking.
- ☐ Entendo como índices B+ funcionam e sei criá-los com JPA (@Index).
- ☐ Consigo usar @Cacheable, @CachePut e @CacheEvict corretamente.
- ☐ Implementei LRU Cache com LinkedHashMap e sei quando usá-lo vs Redis.
- ☐ Sei configurar Exchange, Queue e Binding no RabbitMQ.
- ☐ Implementei Rate Limiter com Sliding Window e integrei ao Spring.
- ☐ Implementei Trie com autocompletar e entendo sua complexidade.
- ☐ Sei escolher a estrutura correta para cada cenário de API REST.
- ☐ Tenho o esqueleto do Projeto Integrador (EduMetrics API) pronto.

Estimativa de tempo por trilha:

Trilha 1 — JMH e Benchmarking	60 minutos
Trilha 2 — Árvores B+ e JPA	45 minutos
Trilha 3 — Cache e Redis	60 minutos
Trilha 4 — RabbitMQ e Padrões	60 minutos
Trilha 5 — Integração e Projeto	75 minutos
TOTAL	300 minutos (5 horas)